Static Analysis on the Entire Linux Kernel?

→ A History and Analysis

Anamitra Ghorui
<aghorui (at) cse.iitb.ac.in>

Uday Khedker
<uday (at) cse.iitb.ac.in>

PLATO CSE IIT Bombay

[IICT 2025] © PLATO Research Group @ CSE, IITB

Static Analysis

- Static Analyses form an important part of performing optimizations and detecting errors in programs.
- Many software packages exist for static code analysis such as CodeQL, SonarQube, LLVM, LSP, IDE tools etc.
- Many conventional languages can only be very poorly statically analysed due to scalability issues and features inherent to the language design: Javascript, Python, Java
- **Analyzing entire programs as a single unit, or Whole-Program Analysis is an even more challenging task.**

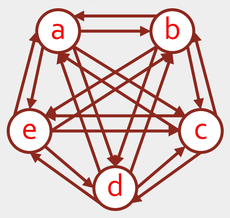
Bugs a pointer analysis can detect:

Use After Free	Dangling Pointers	Null Pointer Deref.
OOB Access	Buffer Overflow	Information Leaks	...

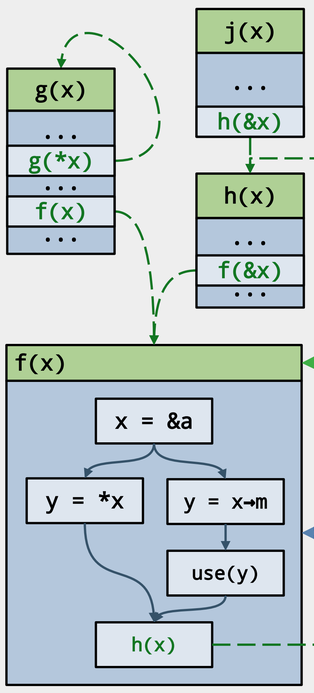
Pointer Analysis (PTA)

Pointer Analysis is a critical requirement, a beneficiary of, and a challenge for Whole-Program Analysis.

- Pointer Analysis is essential for generating **accurate callgraphs** in programs, and therefore essential for a **correct whole-program analysis**.
- Unlike other analyses, (liveness, const. prop.) information in pointer analysis increases **non-linearly** or combinatorially with the amount of variables.
- This causes blow-up of information when analysis is performed naively in large programs, i.e. **not scalable**.
- Making a precise, scalable pointer analysis is an unsolved problem.



The Metrics



SOUNDNESS

Does the analysis result represent at least all of the paths a program can take?

An unsound analysis is not useful for program optimization.

PRECISION

Does the analysis result represent at most all of the paths a program can take? If not, how many spurious paths are there?

We measure this with two main metrics (not necessarily exhaustive):

CONTEXT SENSITIVITY

Does the analysis consider the context in which the function is called (parameters, global state, location, etc.)?

For example, `doSomething(3, "apple")` may have a very different result from `doSomething(4, "ball")`

FLOW SENSITIVITY

Does the analysis consider in what order are instructions executed?

An analysis may simply return what is true across all instructions as a compromise for efficiency.

SCALABILITY

Does the analysis finish on a given program with a reasonable amount of time and resources?

This is important especially for analyzing large programs.

What can you do with...

An Unsound Analysis?

- Detect bugs and vulnerabilities.
- IDE limited refactoring tools.
- Profiling and code-metric analysis tools.

A Sound Analysis?

- Program transformation.
- Program optimizations.
- Analysis and refactoring tools that perform less erroneously.

A Precise Analysis?

- **Useful (?)** program transformation and optimizations.
- Analysis and refactoring tools that perform accurately.
- High-accuracy memory error and race detection. Shape analysis.

Timeline and Past Attempts

Legend: - Available - Partially Available - Absent

1994

Andersen, Lars Ole. – Andersen's Analysis

- First widely recognised Pointer Analysis Algorithm.
- Flow and context sensitive variants exist but are not scalable without caveats.

1996

Steensgaard, Bjarne. – Steensgaard's Analysis

- Modification of Andersen's analysis to be faster but at expense of more precision.
- Implemented in frameworks like LLVM.

2002 — LSM Only

Zhang et. al. – Using CQUAL for Static Analysis

- Used CQUAL: a type based inference system to **interprocedurally** detect usage of resources without proper authorisation in **Linux Security Modules (LSM)**.
- Performed Modifications to GCC, added annotations to declarations to detect vulnerabilities.

2003 — Limited Interprocedural Behaviour

Linus Torvalds – Sparse

- Annotation-based static checker tool initially meant for the kernel. Annotations may be address spaces, value ranges, scope, and other semantics.
- Builds a minimal AST from annotations to perform checking and analysis.

2007 — Whole-Program

Anderson et al. – Deputy, CCount, BlockStop

- Combination of 3 tools: **Deputy** – Annotation based type checker, **CCount** – Runtime refcount based allocation checker, **Blockstop** – Callgraph checker for detecting invalid blocking function calls.
- Uses an "overly conservative" points-to analysis for function pointers.

2017 — Whole-Program

Wang et al. – Graspan

- Converts pointer analysis to a graph reachability problem with a control-flow summarizing expression graph. Used for vulnerability detection primarily.
- Uses a set of worker threads and information disk-backing to make long-running and large analysis possible.
- Context sensitivity achieved through aggressive inlining in graphs. Analysis not claimed to be unsound.

2018 — Drivers Only

Machiry et al. – DR CHECKER

- Implements a flow and context-sensitive analysis for kernel drivers. Soundness guarantee is claimed given some critical assumptions, e.g. all core kernel functions are correctly implemented.
- Preanalysis is done on drivers to find entry points. Work and implementation mainly focuses on taint analysis and detecting vulnerabilities.

2018 — Syscalls Only

Gens et al. – K-Miner

- Syscalls used as entry points. Builds a new global context for each syscall before interprocedural analysis. Flow and context-sensitive analysis using many techniques: parallelization, flow-insensitive preanalysis etc.
- Soundness is unproven but an attempt is made to keep results sound as possible. Analysis takes days to run but takes <= 32 GB of memory.

2024 — Syscalls and related components

Liang et al. – K-LEAK

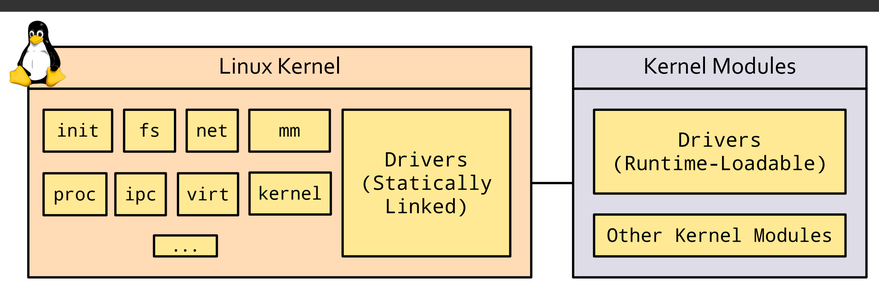
- Uses custom data-flow value called memory-augmented DFG to infer further possible memory errors with fuzzer (syzbot) bug reports/repros as starting point. Builds on previous analysis framework (SUTURE).
- Includes heavy heuristic-based summarization of paths and contexts.

The Linux Kernel

40,098,456

Lines of code in the kernel as of Jan 2025

- The Linux kernel is **written in C**. This makes it prone to eg. pointer vulnerabilies due to manual memory management.
- The kernel has **thousands** of developers contributing to it from individual, volunteer contributors to industry-sponsored ones. All have different levels of skill and knowledge.
- Majority of the code (60%) is in the form of **drivers**.
- Static analysis of Linux code therefore is very important for bug/vulnerability Detection and optimization.

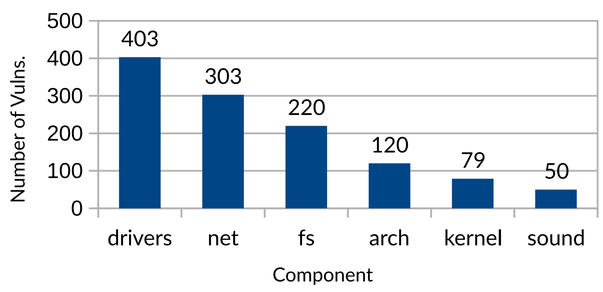


Why is Analyzing the Kernel Hard?

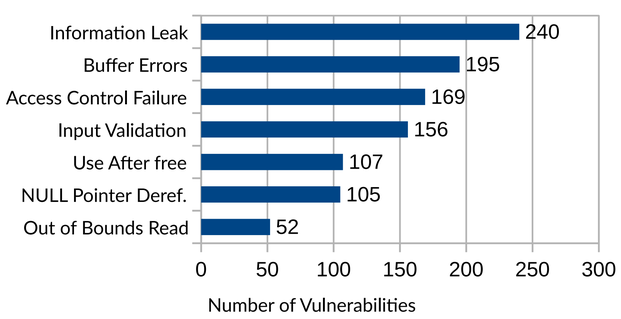
- **Extremely Large Code-base**: Designing useful, scalable analyses is a problem.
- **Uncommon Program Semantics**: Custom allocators, interrupt mechanisms, global state etc.
- **Uncommon Code Linking Methods**: Drivers and other modules may be statically linked or loaded at runtime as "kernel objects."
- **Heterogeneous code and Inline Asm.**: Multiple languages make the kernel harder to analyse.

Kernel Vulnerabilities

- **Taint and Pointer-based vulnerabilities** make up by far the largest share of vulnerabilities in the kernel.
- The most vulnerabilities lie in the **drivers section** of the kernel, which is also the place where **more non-core developers** contribute to the kernel who are likely to be less experienced.
- It therefore makes sense to have analyses that can detect the erroneous interprocedural propagation of values and invalid pointer and resource usage within the program.



Data from 1858 vulerabilities between Jan 2010 and Jan 2020



Data from 1858 vulerabilities between Jan 2010 and Jan 2020

Key Challenges and Takeaways

- **Precise, sound and scalable pointer analysis, and by extension whole-program analysis remains an unsolved problem to this day.**
- Works on pointer analysis still struggle to tackle the problems of **flow sensitivity and context sensitivity** in interprocedural analyses.
- However, **partially sound or "soundy" analyses** have turned out to be of great use in domains other than compiler optimization, and research trends continue to go in this direction.
- Taint analysis is one of the main applications of recent pointer analysis work.
- Whole-program analysis on large, nontrivial programs like Linux require proper sourcecode/IR annotations, pre-analysis initialisations and knowledge about the program itself to perform successfully, eg. syscalls, entry points, linkage methods etc.

How Useful is a Precise Pointer Analysis?

- **This is an unanswered question,** however both recent and older works corroborate about our claim of its utility. However, as we see above, no one has **yet** built an efficient flow and context sensitive whole program pointer analysis.
- Recent work on LLVM shows that imprecise alias information has little advantage over having no info at all. There also seems to be evidence of optimization passes in LLVM do not use the info correctly, resulting in **less efficient programs, so it may be better to not use PTA-based passes at all in such cases.**
- **There is ongoing work at IIT Bombay towards efficient pointer analysis & finding an answer to this question and other dilemmas.**

Refs. and Atributions: https://tinyurl.com/IICT25WPLinux

© 2025 Anamitra Ghorui, Uday Khedker BY-NC-SA